

Design and Analysis of Algorithm

Course Code : SE-2205

Lecture 1

Department of Software Engineering

Course Objectives

- This course introduces students to the analysis and design of computer algorithms. Upon completion of this course, students will be able to do the following:
 - Analyze the asymptotic performance of algorithms.
 - Show a understanding with major algorithms and data structures.
 - Apply important algorithmic design examples and methods of analysis.

Lecture #	Lecture Contents	Task
1.	Introduction and Asymptotic Notations	
2.	Mathematical Tools and Application	
3.	Amortized analysis	
4.	Inductive tools and algorithm correctness	Quiz 01, Assignment 01
5.	Minimum Cost Spanning trees	
6.	Divide and Conquer	
7.	Greedy method	
8.	Decrease-and-Conquer	
	MIDTERM	
9.	Dynamic Programming I	
10.	Dynamic Programming II	
11.	BacktrackingI	
12.	BacktrackingII	Quiz 02, Assignment 02
13.	Branch And Bound Techniques I	
14.	Branch And Bound Techniques II	
15.	NP Complete Problems I	
16.	NP Complete Problems II	

FINAL EXAMS

Course Information

- Office hours
 - 9:00-01:00 Monday to Friday
- Grading policy
 - Assign and Quiz. 20%, Midterm: 30%, Final: 50%

More Information

- Textbook

- *Introduction to The Design and Analysis of Algorithms 2nd Edition*, Anany Levitin, Addison-Wesley, 2007.

- Others

- *Introduction to Design & Analysis Computer Algorithm 3rd*, Sara Baase, Allen Van Gelder, Addison-Wesley, 2000.
- *Algorithms*, Richard Johnsonbaugh, Marcus Schaefer, Prentice Hall, 2004.
- *Introduction to Algorithms*, Cormen, T. H., Leiserson, C.E., Rivest, R.L. & Stein, C (CLRS)., 3rd Edition, MIT Press, 2009

An Algorithm

- An Algorithm is a **finite sequence of instructions**, each of which has a clear meaning and can be performed with a finite amount of effort in a finite length of time. No matter what the input values may be, an algorithm terminates after executing a finite number of instructions. In addition every algorithm must satisfy the following criteria:
- **Input:** there are zero or more quantities, which are externally supplied; **Output:** at least one quantity is produced;
- **Definiteness:** each instruction must be clear and unambiguous;
- **Finiteness:** if we trace out the instructions of an algorithm, then for all cases the algorithm will terminate after a finite number of steps;
- **Effectiveness:** every instruction must be sufficiently basic that it can in principle be carried out by a person using only pencil and paper.

ALGORITHMICS

It is the science that lets designers study and evaluate the effect of algorithms based on various factors so that the best algorithm is selected to meet a particular task in given circumstances. It is also the science that tells how to design a new algorithm for a particular job.

What is a program?

- A program is the expression of an algorithm in a programming language
- a set of instructions which the computer will follow to solve a problem



Characteristics of an algorithm:-

- Must take an input.
- Must give some output(yes/ no, value etc.)
- Definiteness –each instruction is clear and unambiguous.
- Finiteness –algorithm terminates after a finite number of steps.
- Effectiveness –every instruction must be basic i.e. simple instruction.

Let us write down the algorithm for a
problem that is familiar to us

Converting a decimal number into binary

Convert 75 to Binary

2	75	remainder
2	37	1
2	18	1
2	9	0
2	4	1
2	2	0
2	1	0
	0	1



1001011

Algorithm for Decimal-to-Binary Conversion

1. Write the decimal number
2. Divide by 2; write quotient and remainder
3. Repeat step 2 on the quotient; keep on repeating until the quotient becomes zero
4. Write all remainder digits in the reverse order (last remainder first) to form the final result

Points to Note:

1. The process consists of **repeated application** of simple steps
2. All steps are **unambiguous** (clearly defined)
3. We are **capable** of doing all those steps
4. Only a **limited no. of steps** needs to be taken
5. Once all those steps are taken according to the prescribed sequence, the **required result will be found**
6. Moreover, the process will **stop** at that point

Algorithm (Better Definition)

1st Definition:

Sequence of steps that can be taken to solve a problem

Better Definition:

A *precise sequence* of a *limited number* of *unambiguous, executable steps* that *terminates* in the form of a *solution*

Analysis of Algorithms

- Analysis in the **context of algorithms** is concerned with **predicting the resources** that it requires:
 - Computational **time**
 - **Memory**
 - Bandwidth
 - **Logic functions**
- However, **Time** – generally measured in terms of the **number of steps required** to execute an algorithm - is the resource of **most interest**
- By **analyzing several candidate algorithms**, the most efficient one(s) can be **identified**

Time taken by an algorithm?

- Performance measurement or Aposteriori Analysis:
- Implementing the algorithm in a machine and then calculating the time taken by the system to execute the program successfully.
- Performance Evaluation or Apriori Analysis. Before implementing the algorithm in a system. This is done as follows

Selecting Among Algorithms

When choosing among competing, successful solutions to a problem, choose the one which is the least complex

This principle is called the “Ockham’s Razor,” after William of Ockham - famous 13-th century English philosopher

- 1. How long the algorithm takes :-will be represented as a function of the size of
- the input $f(n) \rightarrow$ how long it takes if 'n' is the size of input.
- 2. How fast the function that characterizes the running time grows with the input size. "Rate of growth of running time".
- The algorithm with less rate of growth of running time is considered better.

Looking for Efficiency

- Investment on computing equipment
- Investment on algorithmics

Investment on computing equipment

- Computer solves a particular problem in $10^{-4} \times 2^n$ sec where n is size of instances
 - If $n = 10$, $10^{-4} \times 2^{10} = 0.1024$ sec
 - If $n = 20$, $10^{-4} \times 2^{20} = 104.85$ sec
 - If $n = 30$, $10^{-4} \times 2^{30} = 107374$ sec = 29.82 hours
 - If $n = 38$, $10^{-4} \times 2^{38} = 7635.49$ hours = 254 days
- Purchase of new machine which solves the problem in $10^{-6} \times 2^n$ sec
 - If $n = 38$, $10^{-6} \times 2^{38} = 763.54$ hours = 31 days
 - If $n = 45$, $10^{-6} \times 2^{45} = 11$ years
- No big gain on investment on new machine

Looking for Efficiency

Investment on algorithmics

- A cubic algorithm can solve the problem at very high speed. An old machine can compute the result in $10 \times n^3$ sec.
 - If $n = 10$, $10 \times 10^3 = 10 \text{ sec}$
 - If $n = 20$, $10 \times 20^3 = 80 \text{ sec}$
 - If $n = 30$, $10 \times 30^3 = 270 \text{ sec} = 4.5 \text{ minutes}$
 - If $n > 200$, $10 \times 200^3 = 1 \text{ day}$
- New algorithm offers a much greater improvement than the purchase of new computing equipment
- New algorithm + new computing machine can provide results 100 times faster and enables to solve instances 4 or 5 times bigger than with the new algorithm alone in the same length of time.

What kinds of problems are solved by algorithms?

- Searching (Linear and Non-Linear)
- Sorting (Elementary and Advanced)
- String Processing (Pattern Matching, Parsing, Compression, Cryptography)
- Optimization Problem (Shortest routs, Minimum costs)
- Image Processing(Compression, Conversion, Matching)
- Data Mining Algorithms (Clustering, Cleansing, linear regression, logistic regression, SVM, Decision Tree, Neural Networks, Data Preprocessing, Data Description)
- Mathematical Algorithms (Random Number generator, Matrix Operation etc.)

Types of Algorithmic techniques

- Divide and Conquer
- Greedy Algorithms
- Dynamic Programming
- Heuristics
- Brute Force Technique
- Backtracking

Syntax & Semantics

An algo. is “correct” if its:

- Semantics are correct
- Syntax is correct

Semantics:

The concept embedded in an algorithm (the soul!)

Syntax:

The actual representation of an algorithm (the body!)

WARNINGS:

1. An algo. can be syntactically correct, yet semantically incorrect – very dangerous situation!

2. Syntactic correctness is easier to check as compared with semantic

Algorithm Representation

- Generally, SW developers represent them in one of three forms:
 - Pseudo code
 - Flowcharts
 - Actual code

Pseudo Code

- **Language** that is typically used for writing algorithms
- **Similar to** a programming language, but not as rigid
- The **method of expression most suitable** for a given situation is used:
 - At times, **plain English**
 - At others, a **programming language like** syntax

Flowchart

- A graphical representation of a process (e.g. an algorithm), in which graphic objects are used to indicate the steps & decisions that are taken as the process moves along from start to finish
- Individual steps are represented by boxes and other shapes on the flowchart, with arrows between those shapes indicating the order in which the steps are taken

Flowchart Elements

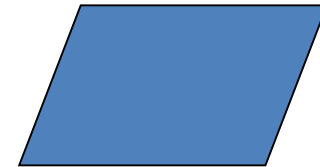
Start or stop



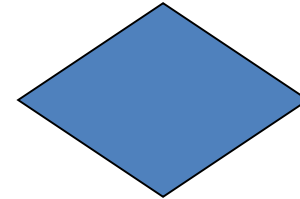
Process



Input or output



Decision



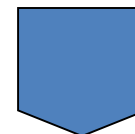
Flow line



Connector



Off-page connector



Any Questions